

3 Example Task 5354 Code Onboarding & Comprehension

Example Task 5354

Name: recharts-recharts-comprehension
Intent: Code Onboarding & Comprehension
Difficulty: Hard
Language: JavaScript

Repo

Repository: <https://github.com/recharts/recharts>

Checkout (commit): 979d921631364d33696d9a5b6f5fba561af5bf5e

Prompt

The src/chart has all the charts but I see they import and then all of them call a function. I am looking to contribute to this library and I thought each chart is its own mini world. But this is confusing me. Can you help me understand why and how this works?

LLM Prompt Review

Prompt QC Feedback

****Grading Item 1 (1/1)****

The question probes a structural pattern across multiple chart modules, which is more complex than a trivial lookup.

****Grading Item 2 (1/1)****

It reads like a natural peer-to-peer inquiry without artificial role-play cues.

****Grading Item 3 (1/1)****

It does not request elementary definitions or basic syntax knowledge.

****Grading Item 4 (1/1)****

Understanding why the charts import and call a shared function requires inspecting repository structure and code flow.

****Grading Item 5 (1/1)****

The prompt is formulated purely as a question and contains no inline code or pasted context.

****Grading Item 6 (0/1)****

The request is quite vague (“why and how this works”) without specifying which function or mechanism, making it difficult to identify a concrete, verifiable answer.

****Grading Item 7 (1/1)****

It aligns with the intent, focusing on understanding repository architecture.

****Grading Item 8 (1/1)****

It invites reasoning about design and behavior rather than a simple factual lookup.

****Grading Item 9 (1/1)****

There are no meta-instructions or formatting directives.

****Grading Item 10 (1/1)****

Only ASCII characters are used.

****Grading Item 11 (1/1)****

The prompt targets a single intent—understanding the architectural pattern for charts.

FINAL_SCORE: 10

Rubric

ID	tag	description	weight	rationale	dependent_on
33993	reasoning	Identifies <code>`generateCategoricalChart`</code> as a factory function	critical	Recognising the factory pattern is the absolute foundation for understanding the codebase and the library	
34002	reasoning	Explains that individual chart components act as thin wrappers or configuration objects	critical	Demonstrates an understanding of dependency injection at the chart level	33993
34005	reasoning	Describes <code>`getDisplayedData`</code> as determining the currently visible slice of data	critical	Data slicing is a core preliminary step before any rendering or coordinate math can occur	
34006	reasoning	States that <code>`getTooltipData`</code> translates raw UI coordinates into resolved tooltip state objects	critical	Understanding the translation of raw DOM events to React state is required for component interactivity	
34007	reasoning	Identifies <code>`getAxisMapByAxes`</code> as the primary function computing complete axis domain configurations	critical	Axis computation is the most complex logic in the file and identifying the primary driver is essential	
34008	reasoning	Notes that <code>`getAxisMapByAxes`</code> takes a calculation shortcut if the user explicitly specifies the axis range	critical	Separating explicit user configuration from implicit data calculation is necessary for a complete understanding	34007

34049	reasoning	Names <code>`getAxisMap`</code> as the router mechanism between the two axis generation strategies	critical	This is the core flow control mechanism for axis generation	
34050	reasoning	Identifies <code>`getAxisNameByLayout`</code> as defining which axis is numeric and which is categorical based on chart layout	critical	This mapping is the foundational truth for layout, stack groups and formatting across the file	
34051	reasoning	Describes <code>`calculateOffset`</code> as isolating the central drawable plotting area	critical	Required to understand how the final rendering boundaries are defined apart from peripheral elements	
34054	reasoning	Explains that <code>`getMouseInfo`</code> translates raw browser screen coordinates into chart relative coordinates	critical	Custom event coordinate translation is a high risk area for interaction bugs and requires thorough understanding	
34055	reasoning	Explains that <code>`getAxisMapByAxes`</code> stretches axis boundaries to accommodate extra markers like reference areas	bonus	Shows comprehensive understanding of how peripheral visual elements affect core chart layout math	
34057	reasoning	Identifies <code>`updateStateOfAxisMapsOffsetAndStackGroups`</code> as the core state recalculation pipeline	critical	This is the engine function that drives the entire data-to-visual transformation sequence	

34058	reasoning	States that <code>`tooltipTicksGenerator`</code> produces arrays sorted by pixel coordinate for efficient nearest tick scanning	bonus	Explains the underlying performance optimization used for the tooltip hover mechanism	
34059	reasoning	Describes <code>`hasGraphicalBarItem`</code> as dynamically determining the use of <code>`scaleBand`</code> along with <code>`scalePoint`</code>	bonus	Identifies how specific chart configurations dynamically alter the underlying mathematical scale functions	
34070	reasoning	Advises trusting the native browser X and Y mouse coordinates directly for tooltips	penalty	Actively bypasses the vital custom <code>`getMouseInfo`</code> translation math, which would cause broken UI bounds	
34071	reasoning	Asserts that <code>`AreaChart`</code> and <code>`BarChart`</code> possess completely distinct rendering logic engines	penalty	Directly contradicts the foundational factory pattern, misleading the developer	
34074	reasoning	Identifies <code>`getAxisMapByItems`</code> as generating a synthetic numeric axis for charts without explicit axis components	critical	Understanding the fallback rendering path is crucial for a complete view	
34075	completeness	Identifies <code>`CategoricalChartWrapper`</code> as the returned React class component	critical	The factory ultimately produces this class and ignoring it misses the entire React integration and lifecycle layer	

34076	completeness	Explains the wrapper component's reliance on a dispatch table ('renderMap') rather than rendering standard JSX children directly	critical	Explaining the custom render tree is vital for auditing how properties are injected into chart children	
34078	completeness	Details the logic inside 'getTooltipContent' for handling duplicate keys via 'findEntryInArray'	bonus	Demonstrates deep inspection of edge cases in the data lookup process	
34079	completeness	Mentions that 'CategoricalChartWrapper' establishes a mouse movement speed limit in its constructor	bonus	Highlights a critical performance and anti crash safe guard built into the UI layer	34075
34107	completeness	Suggests removing or bypassing the Redux 'RechartsStoreProvider' wrapper	penalty	Actively breaks the state management context required by the library at the highest level of the component	
34108	reasoning	Claims that 'getOffset' used by 'getMouseInfo' does not account for any borders on the '<html>' element	penalty	The function also subtracts document.documentElement.clientTop/clientLeft which makes this safe for such rare cases, hence this is wrong.	
34112	style	Formats the explanation using structured hierarchical elements like markdown headers or numbered lists	bonus	Given the sheer volume of functions in the target file, structural formatting is essential for visual parsing	

34113	style	Presents inline code elements and function names using monospace markdown formatting	bonus	Maintains strict technical clarity by distinguishing variable/function names from explanation	
34123	reasoning	Claims that the GraphicalChild passed by every chart component contains the tooltip logic for that chart.	penalty	Tooltip logic is the same for all charts and nothing to do with the GraphicalChild passed by every chart	
34130	completeness	Mentions <code>`calculateActiveTickIndex`</code> as a dependent function called within the <code>`getTooltipData`</code> execution flow	bonus	Shows the understanding of the internal dependency chain of the tooltip coordinate resolution	
34131	completeness	Identifies the specific array method <code>`data.slice()`</code> utilized within <code>`getDisplayedData`</code>	bonus	Highlights the specific and safe JavaScript operation used to isolate the visible data segment	
34140	reasoning	Identifies that <code>`getAxisMapByAxes`</code> adds hidden sequence numbers to deduplicate specific data connections	bonus	It checks you saw how duplicate categories get stable ordering inside <code>getAxisMapByAxes</code> before domain math, which is easy to miss when you only explain the factory.	
34141	reasoning	Mentions that <code>`getTooltipContent`</code> explicitly filters out null entries before returning the final payload array	bonus	Stating that <code>getTooltipContent</code> removes empty series slots from the merged payload so the tooltip list matches real series only, which is finer than the main hover and state pipeline.	
34142	completeness	States that <code>`calculateActiveTickIndex`</code> uses the sorted tick array to perform an efficient left-right comparison scan	bonus	Specific algorithmic approach used for the nearest neighbour hover detection	

34143	reasoning	Notes that the wrapper pre-populates the default tooltip during the mount phase if a `defaultIndex` is provided	bonus	Captures a specific initialization behavior of the interactive tooltip	
35755	completeness	Mentions that `renderByOrder` walks through the JSX children in order applying the right handler from `renderMap`	critical	This function is important to preserve the DOM order while optionally enforcing that certain component types only render once.	
35860	completeness	Mentions that `getMouseInfo` handovers to `handleMouseMove` `handleMouseEnter` or `handleMouseLeave` handlers after getting the coordinates	bonus	Raw browser interactions and peripheral device interactions are natively handled after coordinate computation	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: `reasoning`, `completeness`, and `style`. All three categories are present. (2/2)**

Rubric uses only those three top-level keys.

****Grading Item 2:** The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)**

There are 34 total grading items, which is within the suggested range and reflects the complex golden answer.

****Grading Item 3: Every grading item is atomic and self-contained, evaluating exactly one distinct point. (2/2)****

Each description targets a single behavior or fact; no stacked multi-part checks were found.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)****

Descriptions name concrete functions/behaviors, making evaluation unambiguous.

****Grading Item 5: Every grading item is derivable from the provided `golden\_answer`. (0/2)****

Penalty item `reasoning-34108` references `getOffset` handling `` borders, which the golden answer never mentions, so that requirement is not derivable.

****Grading Item 6: The rubric is comprehensive relative to the `golden\_answer`. (2/2)****

All major concepts from the golden answer—factory pattern, axis builders, tooltip flow, wrapper lifecycle—are represented.

****Grading Item 7: The rubric would award full marks to the provided `golden\_answer`. (2/2)****

The golden answer satisfies every critical item and triggers no penalties, so it would earn full credit.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths. (2/2)****

Although detailed, items allow any repo-grounded explanation covering the same mechanics, not just the golden answer's wording.

****Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment, conceptual understanding, and explanation. (2/2)****

Reasoning items all target understanding of internal mechanisms and why they behave as described.

****Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts of the question. (2/2)****

Completeness items focus on covering structural/rendering flow pieces required by the prompt.

****Grading Item 11: The `style` category evaluates clarity, organization, readability, and professionalism. (2/2)****

Style items concern formatting and code-style presentation, not technical content.

****Grading Item 12: Grading items are placed in appropriate categories. (2/2)****

Each item aligns with its category's focus; no misplaced requirements observed.

****Grading Item 13: The rubric includes the task-wide expectations that should always be present: reasoning quality, completeness, and code style. (2/2)****

Each of the three categories contains relevant grading items covering these expectations.

****Grading Item 14: Intent-specific grading items are included when required. (2/2)****

The task is an architectural explanation; rubric items appropriately cover factory mechanics, tooltips, and axis logic without omitting necessary intent-specific checks.

****Grading Item 15: Weights use only valid values and are well calibrated. (2/2)****

Weights are limited to `critical`, `bonus`, or `penalty`, and their usage matches the importance of each requirement.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior and are not simple negations. (2/2)****

All penalties call out concrete harmful misconceptions rather than just negating positive requirements.

****Grading Item 17: Dependency usage is logically sound. (2/2)****

Dependencies appear only where one item relies on another prerequisite, and the directions are appropriate.

****Grading Item 18: The rubric is not overprescriptive. (1/2)****

While comprehensive, the large number of critical items tied to very specific functions risks penalizing otherwise solid explanations that focus on higher-level architecture, making it somewhat overprescriptive.

****Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)****

Items cover distinct, valuable aspects of understanding the chart factory, with minimal redundancy.

FINAL_SCORE: 35

Golden Answer

`AreaChart`, `BarChart`, `LineChart` etc all depend on a factory function `generateCategoricalChart`. This function takes a chart configuration as an argument and returns a React component `CategoricalChart`. Simply put, all the charts are just thin wrappers around the component this function produces.

This means, every chart type is just a configuration, not a separate codebase. All the charts share 100% of the same logic defined once. Each chart only declares what makes it unique. Without the factory, adding a bug fix would require changing separate chart files instead of one. The factory is essentially dependency injection at the chart level.

The file `src/chart/generateCategoricalChart.tsx` has all the required code along with some imports for helpers and utils. It exports among all the other exports, `generateCategoricalChart` which is further imported by all the individual chart files. The file can be broken down into the following major sections:

1. Imports and Constants (L1 - L125)
2. Data Helpers (L143 - L260)

3. Axis Map Builders (L275 - L613)
4. State Initialiser (L621 - L647)
5. Layout Helpers (L649 - L752)
6. Types and Props (L754 - 800)
7. Factory Function (L802 - L1965)

Important Helper and Build Functions

1. ``getDisplayedData`` (L143) determines which slice of data is currently visible in the chart, respecting two different data sources and the brush window. It is used by functions like ``getAxisMapByAxes`` to know how many data points are visible on this axis, inside ``getFormatItems`` to pass the right displayed data to each series and inside ``getTooltipContent`` to look up the payload at the active index. This function uses ``data.slice()`` which is safe.

2. ``getTooltipContent`` (L179) builds an array of tooltip payload entries, one per graphical series, for the currently hovered data point. It first calls ``getDisplayedData`` to get the current visible data slice, checks for ``activeIndex`` returning ``null`` if invalid, and for each graphical item:

- a. Determines which data source to use for that item.
- b. Applies the brush window slice to that data.
- c. Uses ``findEntryInArray`` to look up by ``activeLabel`` value if duplicates are not allowed or uses ``activeIndex`` if duplicates are allowed or there is no ``dataKey``
- d. Calls ``getTooltipItem(child, payload)`` to build the ``{name, value, color, dataKey, ...}`` entry.

It finally filters out any ``null`` entries.

This function is called by ``getTooltipData`` during mouse movements to build the full tooltip state, ``applySyncEvent`` when a synced chart broadcasts its tooltip position to this chart and ``displayDefaultTooltip`` to pre populate the tooltip at mount with ``defaultIndex``.

3. ``getTooltipData`` (L233) converts a raw mouse or touch coordinate into a fully resolved tooltip state object. It first uses the ``rangeObj {x, y}`` (or ``state.chartX`` and ``state.chartY``) and passes them to ``calculateTooltipPos(rangeData, layout)`` which extracts the single relevant coordinate (x for horizontal, y for vertical, angle for polar). The return value is a number which is passed to ``calculateActiveTickIndex`` that calculates the ``activeIndex`` closest to the ``pos``. If ``activeIndex`` is zero or greater (and ``tooltipTicks`` is present), ``activeLabel``, ``activePayload`` and ``activeCoordinate`` are fetched and returned, else ``null`` is returned. It is called by ``getMouseInfo`` and ``getDerivedStateFromProps``.

4. ``getAxisMapByAxes`` (L275) is the primary axis configuration builder. Given a set of ``<XAxis>`` or ``<YAxis>`` React elements, it produces a map of ``axisId`` which is complete axis config object with computed domain. This is the most complex function in the file. It uses a decision tree to compute ``domain`` for each axis element which works as follows:

- a. First, it checks if the user explicitly told it what the axis range should be. If yes, it takes a performance shortcut by just using what the user specified and completely skips scanning the actual data.

- b. If the above step fails (user did not explicitly tell), then it has to figure it out from the data. To do this, it checks whether the axis is tied to a specific piece of data:

- i. If it has a specific data connection: The function does some deduplication by adding hidden sequence numbers, filters out empty or null values and check if the data includes error bars and stretches the axis limits so they don't get cut off.

- ii. If it does not have a specific data connection: It defaults to assigning basic sequence numbers based on how many items are there, calculates the total range needed to fit the combined, stacked groups (if any) and it defaults to looking at all the chart items sharing that axis to find the lowest and highest values.

- c. Once the system has calculated the core range, it does two final checks before drawing the axis:

- i. It checks if you added extra markers, like a target line or a shaded reference area and if these fall outside the current range, it stretches the axis boundaries to make sure they are visible.

- ii. It applies any specific minimum or maximum limits the user might have provided to fine tune the final result.

5. ``getAxisMapByItems`` (L459) is the fallback axis builder. It is used when the user adds ``<Line>`` or ``<Bar>`` but does not add any explicit ``<XAxis>`` or ``<YAxis>`` children. It creates a synthetic, hidden axis configuration based purely on what the graphical items need. They are never visually rendered, just used for scale computation. The key difference with ``getAxisMapByAxes`` is ``getAxisMapByItems`` does not try to compute any complex domain types, but it just needs a numeric domain that is good enough to render the chart without explicit axis elements.

6. ``getAxisMap`` (L555) is the single entry point for axis map construction. It is a router between the two strategies ``getAxisMapByAxes`` and ``getAxisMapByItems``. It simply checks if the axes are present and routes the rendering of the axis to either of the functions. If no graphical items also exists, it simply returns a blank object.

7. ``tooltipTicksGenerator`` (L605) extracts tooltip tick coordinates from the categorical axis map. It returns two arrays, ``tooltipTicks`` which preserves the data order index and

`orderedTooltipTicks` which is sorted by pixel coordinate, enabling
`calculateActiveTickIndex` to do an efficient left-right comparison scan to find the
nearest tick to the mouse. The result is merged into component state in
`updateStateOfAxisMapsOffsetAndStackGroups`. The three fields are read directly in
`getTooltipData` and `applySyncEvent`.

8. `createDefaultState` (L621) creates a clean initial state object for the chart. It is called
on the first render and whenever the chart needs to fully reset.

9. `hasGraphicalBarItem` (L649) returns `true` if any graphical item's display name
contains "Bar" i.e. if the chart has any `` or `` component. Bar charts
need `scaleBand` where each category takes equal space, so this function is used to
determine if `scaleBand` is needed or `scalePoint` does the job.

10. `getAxisNameByLayout` (L661) returns a `{numericAxisName, cateAxisName}` pair
telling the rest of the code, which axis role is which, based on chart layout. The entire
axis map construction, stack group computation and `getFormatItems` rely on knowing
which axis carries numeric data and which carries categories. This function is the single
source of truth of that mapping, changing it would flip axes for the whole chart. It is
called at the very top of both `updateStateOfAxisMapsOffsetAndStackGroups` and
`getFormatItems`.

11. `calculateOffset` (L684) is the last function before the main factory function in this
file. It calculates the drawable area of the chart which is the rectangle inside which data
is actually plotted by subtracting all peripheral elements from the total chart dimensions.

`generateCategoricalChart` Factory Function

The function receives the following configuration options:

```
``ts
{
  chartName,          // e.g. "AreaChart", "BarChart"
  GraphicalChild,     // e.g. Area, Bar, Line
  defaultTooltipEventType, // 'axis' or 'item'
  validateTooltipEventTypes,
  axisComponents,     // axis types this chart supports
  formatAxisMap,      // function to finalize axis scales
  defaultProps,       // chart-specific prop defaults
  tooltipPayloadSearcher
}
```

...

There are functions within and some important ones are described as follows:

1. ``getFormatItems`` (L812) loops through each graphical child and computes its display data like positions, stacked data, bar sizes, tick arrays by calling ``item.type.getComposedData()``. The results are stored as ``formattedGraphicalItems`` in state.
2. ``updateStateOfAxisMapsOffsetAndStackGroups`` (L934) is the core state recalculation pipeline. It runs whenever data or layout or children change.
 - a. Finds all graphical items.
 - b. Computes stack groups
 - c. Builds axis maps
 - d. Calculates the chart offset.
 - e. Applies ``formatAxisMap`` to finalise D3 scales on each axis.
 - f. Generates tooltip ticks
 - g. Calls ``getFormatItems`` to get the final positioned data for each series.
3. ``CategoricalChartWrapper`` (L1020) is the React class component. It is the main chart component. The lifecycle is as follows:
 - i. ``constructor`` and ``componentWillUnmount``: When the chart is first created, it sets up an ID for cropping the chart area and creates a speed-limit for mouse movements so the browser doesn't crash when the cursor is moved around. When the chart is removed, it cleans up those listeners so it does not leave junk behind in your browser's memory.
 - ii. ``getDerivedStateFromProps``: This is the engine. Whenever the data changes, window is resized, or a prop is tweaked, this method catches it. It recalculates everything, where the axes go, how things stack, and what needs to change.
 - iii. ``componentDidMount``, ``componentDidUpdate``, ``getSnapshotBeforeUpdate``: These handle the finer details. They make sure the default tooltip pops up, keep the accessibility tools (like screen readers) in sync with visual changes, and listen for syncing events from other charts.

When a click, touch or drag events across the chart are detected, all of that action funnels through one master method called ``getMouseInfo()``. Instead of trusting the browser's raw X/Y coordinates, ``getMouseInfo()`` does its own math. It grabs the exact boundaries of the chart container, translates screen coordinates into chart coordinates, checks to make sure the cursor is actually inside the active chart area. It finds the nearest tick mark on the axis and figures out what data point belongs to it if the hover is

over a general chart or it grabs the raw coordinates if hovering over a specific bar or dot. It then hands it off to the right handler, `handleMouseMove`` `handleMouseEnter`` or `handleMouseLeave``.

Rather than rendering `{children}` directly, the component uses a dispatch table (`renderMap``). `renderByOrder`` walks the JSX children in order and applies the right handler from this map.

The factory ultimately returns this thin functional wrapper that:

1. Creates a Redux store options (`ChartOptions``)
2. Wraps `CategoricalChartWrapper`` in `RechartsStoreProvider``

Golden Answer Review

Golden Answer QC Feedback

****Grading Item 1:**** The `golden_answer`` is internally consistent and technically accurate. It does not contradict itself, make unsupported claims, or rely on incorrect technical reasoning. If code snippets or concrete fixes are included, they should be coherent and should not obviously reference undefined variables, missing imports, or broken logic. (3/3)

All described behaviors and functions align logically, with no contradictions or implausible mechanics detected.

****Grading Item 2:**** The `golden_answer`` is clearly grounded in repo exploration rather than generic software advice. It references specific files, functions, modules, call flows, code locations, or concrete repo behaviors where appropriate. (3/3)

It cites `src/chart/generateCategoricalChart.tsx``, enumerates line ranges, and explains concrete functions/calls throughout the repo.

****Grading Item 3:**** The ``golden_answer`` directly answers the question asked in the ``prompt_statement`` without drifting into unrelated explanation or omitting the central conclusion. (2/2)

It explicitly clarifies why charts share a factory and how that structure works, matching the prompt.

****Grading Item 4:**** The ``golden_answer`` explains the reasoning chain behind its conclusion. It shows how the answer follows from the observed repo behavior, architecture, data flow, control flow, change impact, or causal chain, rather than simply stating conclusions. (3/3)

Each described function includes causal explanations of how data flows and why behaviors occur.

****Grading Item 5:**** Technical explanations are precise and mechanistically accurate, avoiding vague phrases like "causes issues" or "creates confusion" when a concrete explanation of what happens is needed. (2/2)

Mechanisms (e.g., tooltip resolution, axis range shortcuts) are spelled out concretely.

****Grading Item 6:**** Technical details are factually accurate for the repo context, framework behavior, language behavior, and any stated versions or assumptions. (2/2)

Descriptions match Recharts' architecture and do not introduce incorrect assumptions.

****Grading Item 7:**** The `golden\_answer` satisfies all critical grading items in the associated rubric. Every must-have requirement identified in the rubric is addressed. (3/3)

All critical reasoning/completeness points (factory pattern, axis routing, wrapper class, renderMap, etc.) are covered.

****Grading Item 8:**** The `golden\_answer` avoids all penalty behaviors identified in the associated rubric. It does not include harmful, misleading, or clearly incorrect content that would trigger penalty grading items. (2/2)

No flagged penalty statements (e.g., distinct logic engines, bypassing store, trusting raw coords) appear.

****Grading Item 9:**** The `golden\_answer` is appropriately comprehensive for the prompt. It covers the major required points without being so thin that important reasoning or implications are missing. (2/2)

It offers broad coverage of helpers, factory, wrapper, and interaction flow.

****Grading Item 10:**** The `golden\_answer` remains understandable to a non-expert reader. It is clear, organized, and professional, and where the task calls for explanation-oriented writing, it uses instructive language that helps a new engineer follow the reasoning. (2/2)

Complex content is organized via sections and plain-language explanations.

****Grading Item 11:**** Where appropriate, the `golden\_answer` includes concrete supporting details rather than only abstract statements. Examples may include file paths, function names, call sequences, inline code snippets, configuration details, or exact code-level implications. (2/2)

It references specific functions, line ranges, configuration objects, and call flows.

****Grading Item 12:**** The `golden\_answer` follows the intent-specific expectations for the task. (3/3)

As a repo comprehension response, it emphasizes architecture and behavior explanations without proposing fixes, satisfying intent.

****Grading Item 13:**** Not applicable (non-PR task). (2/2)

Automatically awarded full credit.

****Grading Item 14:**** The `golden\_answer` does NOT contain non-ASCII characters except when necessary. (1/1)

Only ASCII characters are used.

FINAL_SCORE: 32

Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-34071",
      "tag": "reasoning",
      "weight": "penalty",
      "description": "Asserts that `AreaChart` and `BarChart` possess completely distinct rendering logic engines",
      "rationale": "Directly contradicts the foundational factory pattern, misleading the developer",
      "dependent_on": [],
      "pass": true,
      "evidence": "\"All the charts share 100% of the same logic defined once.\""
```


"reasoning": "The response explicitly states that AreaChart and BarChart reuse the same factory-generated logic, directly contradicting any notion that they have separate rendering engines."

},

{

"id": "reasoning-34055",

"tag": "reasoning",

"weight": "bonus",

"description": "Explains that `getAxisMapByAxes` stretches axis boundaries to accommodate extra markers like reference areas",

"rationale": "Shows comprehensive understanding of how peripheral visual elements affect core chart layout math",

"dependent_on": [],

"pass": true,

"evidence": "it checks if you added extra markers, like a target line or a shaded reference area and if these fall outside the current range, it stretches the axis boundaries to make sure they are visible.",

"reasoning": "The golden answer explicitly states that `getAxisMapByAxes` adjusts axis boundaries when extra markers such as reference areas lie outside the initial range, satisfying the criterion."

},

{

"id": "reasoning-34049",

"tag": "reasoning",

"weight": "critical",

"description": "Names `getAxisMap` as the router mechanism between the two axis generation strategies",

"rationale": "This is the core flow control mechanism for axis generation",

"dependent_on": [],

"pass": true,

"evidence": "`getAxisMap` is the single entry point for axis map construction. It is a router between the two strategies `getAxisMapByAxes` and `getAxisMapByItems`.",

"reasoning": "The answer explicitly states that `getAxisMap` serves as the routing mechanism between the two axis generation strategies, directly satisfying the criterion."

},

{

"id": "reasoning-34008",

"tag": "reasoning",

"weight": "critical",

"description": "Notes that `getAxisMapByAxes` takes a calculation shortcut if the user explicitly specifies the axis range",

"rationale": "Separating explicit user configuration from implicit data calculation is necessary for a complete understanding",

"dependent_on": [

"reasoning-34007"

],

"pass": true,

"evidence": "\u201ca. First, it checks if the user explicitly told it what the axis range should be. If yes, it takes a performance shortcut by just using what the user specified and completely skips scanning the actual data.\u201d",

"reasoning": "The answer explicitly states that getAxisMapByAxes bypasses data scanning when a user supplies an axis range, demonstrating the required understanding."

},

{

"id": "reasoning-34002",

"tag": "reasoning",

"weight": "critical",

"description": "Explains that individual chart components act as thin wrappers or configuration objects",

```

    "rationale": "Demonstrates an understanding of dependency injection at the chart
level",
    "dependent_on": [
        "reasoning-33993"
    ],
    "pass": true,
    "evidence": "\u201cAreaChart, BarChart, LineChart etc all depend on a factory
function `generateCategoricalChart`. \u2026 all the charts are just thin wrappers around
the component this function produces.\u201d",
    "reasoning": "The answer explicitly states that each chart is a thin
wrapper/configuration around the generated component, directly fulfilling the criterion."
},
{
    "id": "reasoning-34059",
    "tag": "reasoning",
    "weight": "bonus",
    "description": "Describes `hasGraphicalBarItem` as dynamically determining the use
of `scaleBand` and `scalePoint`,
    "rationale": "Identifies how specific chart configurations dynamically alter the
underlying mathematical scale functions",
    "dependent_on": [],
    "pass": true,
    "evidence": "`hasGraphicalBarItem` ... returns `true` if any graphical item's display
name contains `Bar` ... Bar charts need `scaleBand` where each category takes equal
space, so this function is used to determine if `scaleBand` is needed or `scalePoint`
does the job.",
    "reasoning": "The answer explicitly explains that `hasGraphicalBarItem` detects bar
components and determines whether to use `scaleBand` versus `scalePoint`, satisfying
the criterion."
},

```

```
{
  "id": "reasoning-34005",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Describes `getDisplayedData` as determining the currently visible slice
of data",
  "rationale": "Data slicing is a core preliminary step before any rendering or coordinate
math can occur",
  "dependent_on": [],
  "pass": true,
  "evidence": "`getDisplayedData` (L143) determines which slice of data is currently
visible in the chart, respecting two different data sources and the brush window.",
  "reasoning": "The answer explicitly states that `getDisplayedData` determines the
currently visible data slice, directly satisfying the criterion."
},
{
  "id": "reasoning-34057",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `updateStateOfAxisMapsOffsetAndStackGroups` as the
core state recalculation pipeline",
  "rationale": "This is the engine function that drives the entire data-to-visual
transformation sequence",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cupdateStateOfAxisMapsOffsetAndStackGroups` (L934) is the
core state recalculation pipeline. It runs whenever data or layout or children
change.\u201d",
```

```
"reasoning": "The golden answer explicitly labels  
`updateStateOfAxisMapsOffsetAndStackGroups` as the core state recalculation  
pipeline and describes its role, directly satisfying the criterion."  
},  
{  
  "id": "reasoning-34108",  
  "tag": "reasoning",  
  "weight": "penalty",  
  "description": "Claims that `getOffset` used by `getMouseInfo` does not account for  
any borders on the `` element",  
  "rationale": "The function also subtracts  
document.documentElement.clientTop/clientLeft which makes this safe for such rare  
cases, hence this is wrong.",  
  "dependent_on": [],  
  "pass": true,  
  "evidence": "The golden answer discusses `getMouseInfo()` but makes no mention  
of `getOffset`, nor does it claim that `getOffset` fails to account for borders on the  
`<html>` element.",  
  "reasoning": "Since the golden answer does not assert the incorrect behavior  
regarding `getOffset`, it avoids the penalized claim."  
},  
{  
  "id": "reasoning-34006",  
  "tag": "reasoning",  
  "weight": "critical",  
  "description": "States that `getTooltipData` translates raw UI coordinates into  
resolved tooltip state objects",  
  "rationale": "Understanding the translation of raw DOM events to React state is  
required for component interactivity",  
  "dependent_on": [],
```

"pass": true,

"evidence": "\u201c`getTooltipData` (L233) converts a raw mouse or touch coordinate into a fully resolved tooltip state object. It first uses the `rangeObj {x, y}`\u2026 and passes them to `calculateTooltipPos`\u2026 calculates the activeIndex\u2026 `activeLabel`, `activePayload` and `activeCoordinate` are fetched and returned, else `null` is returned.\u201d",

"reasoning": "The answer explicitly explains that `getTooltipData` takes raw mouse/touch coordinates and translates them into the tooltip state, describing the steps involved in this conversion. Therefore it clearly meets the criterion."

},

{

"id": "reasoning-34051",

"tag": "reasoning",

"weight": "critical",

"description": "Describes `calculateOffset` as isolating the central drawable plotting area",

"rationale": "Required to understand how the final rendering boundaries are defined apart from peripheral elements",

"dependent_on": [],

"pass": true,

"evidence": "\u201c`calculateOffset` (L684)\u2026 calculates the drawable area of the chart which is the rectangle inside which data is actually plotted by subtracting all peripheral elements from the total chart dimensions.\u201d",

"reasoning": "The answer explicitly states that `calculateOffset` computes the central drawable area by subtracting peripheral elements, directly meeting the criterion."

},

{

"id": "reasoning-34123",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that the GraphicalChild passed by every chart component contains the tooltip logic for that chart.",

"rationale": "Tooltip logic is the same for all charts and nothing to do with the GraphicalChild passed by every chart",

"dependent_on": [],

"pass": true,

"evidence": "The answer lists `GraphicalChild` only as part of the configuration object (`GraphicalChild, // e.g. Area, Bar, Line`) and attributes tooltip handling to functions like `getTooltipContent`, `getTooltipData`, and `getMouseInfo`, showing tooltip logic is shared rather than tied to `GraphicalChild`.",

"reasoning": "Because the golden answer never claims that tooltip logic resides in the `GraphicalChild`, and instead explains tooltip behavior comes from shared helper functions, it does not violate the penalty criterion."

},

{

"id": "reasoning-34054",

"tag": "reasoning",

"weight": "critical",

"description": "Explains that `getMouseInfo` translates raw browser screen coordinates into chart relative coordinates",

"rationale": "Custom event coordinate translation is a high risk area for interaction bugs and requires thorough understanding",

"dependent_on": [],

"pass": true,

"evidence": "\u201cWhen a click, touch or drag events across the chart are detected, all of that action funnels through one master method called `getMouseInfo()`. Instead of trusting the browser's raw X/Y coordinates, `getMouseInfo()` does its own math. It grabs the exact boundaries of the chart container, translates screen coordinates into chart coordinates\u2026\u201d",

"reasoning": "The answer explicitly explains that `getMouseInfo` converts browser screen coordinates into chart-relative coordinates, directly satisfying the criterion."

},

{

"id": "reasoning-34074",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies `getAxisMapByItems` as generating a synthetic numeric axis for charts without explicit axis components",

"rationale": "Understanding the fallback rendering path is crucial for a complete view",

"dependent_on": [],

"pass": true,

"evidence": "\u201c`getAxisMapByItems` (L555) is the fallback axis builder. It is used when the user adds `` or `` but does not add any explicit `` or `` children. It creates a synthetic, hidden axis configuration based purely on what the graphical items need. ... it just needs a numeric domain that is good enough to render the chart without explicit axis elements.\u201d",

"reasoning": "The golden answer explicitly notes that `getAxisMapByItems` handles cases without explicit axes by generating synthetic numeric axis configurations, fulfilling the criterion."

},

{

"id": "reasoning-34141",

"tag": "reasoning",

"weight": "bonus",

"description": "Mentions that `getTooltipContent` explicitly filters out null entries before returning the final payload array",

"rationale": "Data sanitization process within the tooltip generator is needed to filter any unwanted data",


```
"dependent_on": [],
"pass": true,
"evidence": "It finally filters out any `null` entries.",
"reasoning": "The golden answer explicitly states that `getTooltipContent` filters out
null entries before returning the payload array, demonstrating the required data
sanitization."
},
{
  "id": "reasoning-34050",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `getAxisNameByLayout` as defining which axis is numeric
and which is categorical based on chart layout",
  "rationale": "This mapping is the foundational truth for layout, stack groups and
formatting across the file",
  "dependent_on": [],
  "pass": true,
  "evidence": "10. `getAxisNameByLayout` (L661) returns a `{numericAxisName,
cateAxisName}` pair telling the rest of the code, which axis role is which, based on chart
layout. The entire axis map construction, stack group computation and
`getFormatlItems` rely on knowing which axis carries numeric data and which carries
categories. This function is the single source of truth of that mapping, changing it would
flip axes for the whole chart.",
  "reasoning": "The golden answer explicitly states that `getAxisNameByLayout`
defines which axis is numeric versus categorical depending on the layout and highlights
its foundational role across the file."
},
{
  "id": "reasoning-34058",
  "tag": "reasoning",
```

```
"weight": "bonus",
  "description": "States that `tooltipTicksGenerator` produces arrays sorted by pixel coordinate for efficient nearest tick scanning",
  "rationale": "Explains the underlying performance optimization used for the tooltip hover mechanism",
  "dependent_on": [],
  "pass": true,
  "evidence": "`tooltipTicksGenerator` ... returns two arrays, `tooltipTicks` ... and `orderedTooltipTicks` which is sorted by pixel coordinate, enabling `calculateActiveTickIndex` to do an efficient left-right comparison scan to find the nearest tick to the mouse.",
  "reasoning": "The answer explicitly states that `orderedTooltipTicks` is sorted by pixel coordinate and explains that this sorting allows efficient nearest-tick scanning, satisfying the criterion."
},
{
  "id": "reasoning-34143",
  "tag": "reasoning",
  "weight": "bonus",
  "description": "Notes that the wrapper pre-populates the default tooltip during the mount phase if a `defaultIndex` is provided",
  "rationale": "Captures a specific initialization behavior of the interactive tooltip",
  "dependent_on": [],
  "pass": true,
  "evidence": "It is called by `getTooltipData` during mouse movements ... and `displayDefaultTooltip` to pre populate the tooltip at mount with `defaultIndex`.",
  "reasoning": "The answer explicitly states that during mount, `displayDefaultTooltip` uses `defaultIndex` to pre-populate the tooltip, matching the initialization behavior described in the criterion."
},
```

```
{
  "id": "reasoning-34140",
  "tag": "reasoning",
  "weight": "bonus",
  "description": "Identifies that `getAxisMapByAxes` adds hidden sequence numbers to deduplicate specific data connections",
  "rationale": "These are the data transformation steps which are required for accurate axis plotting, without which duplicates will cause side effects",
  "dependent_on": [],
  "pass": true,
  "evidence": "It states, \"If it has a specific data connection: The function does some deduplication by adding hidden sequence numbers.\",",
  "reasoning": "The golden answer explicitly explains that getAxisMapByAxes adds hidden sequence numbers for deduplication when an axis is tied to a specific data connection, satisfying the criterion."
},
{
  "id": "reasoning-34007",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `getAxisMapByAxes` as the primary function computing complete axis domain configurations",
  "rationale": "Axis computation is the most complex logic in the file and identifying the primary driver is essential",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201c`getAxisMapByAxes` (L275) is the primary axis configuration builder. Given a set of `` or `` React elements, it produces a map \u2026 This is the most complex function in the file.\u201d",
```

"reasoning": "The golden answer explicitly identifies `getAxisMapByAxes` as the primary function responsible for computing complete axis configurations and emphasizes its complexity, satisfying the criterion."

},

{

"id": "reasoning-33993",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies `generateCategoricalChart` as a factory function",

"rationale": "Recognising the factory pattern is the absolute foundation for understanding the codebase and the library",

"dependent_on": [],

"pass": true,

"evidence": "`AreaChart`, `BarChart`, `LineChart` etc all depend on a factory function `generateCategoricalChart`.",

"reasoning": "The golden answer explicitly states that `generateCategoricalChart` is a factory function, which satisfies the criterion that it must identify the function as such."

},

{

"id": "reasoning-34070",

"tag": "reasoning",

"weight": "penalty",

"description": "Advises trusting the native browser X and Y mouse coordinates directly for tooltips",

"rationale": "Actively bypasses the vital custom `getMouseInfo` translation math, which would cause broken UI bounds",

"dependent_on": [],

"pass": true,

"evidence": "\u201cInstead of trusting the browser's raw X/Y coordinates, `getMouseInfo()` does its own math.\u201d",

"reasoning": "The golden answer explicitly rejects relying on native browser coordinates and explains that `getMouseInfo` performs its own coordinate translation, so it does not advise the harmful behavior."

},

{

"id": "completeness-34076",

"tag": "completeness",

"weight": "critical",

"description": "Explains the wrapper component's reliance on a dispatch table (`renderMap`) rather than rendering standard JSX children directly",

"rationale": "Explaining the custom render tree is vital for auditing how properties are injected into chart children",

"dependent_on": [],

"pass": true,

"evidence": "Rather than rendering `{children}` directly, the component uses a dispatch table (`renderMap`). `renderByOrder` walks the JSX children in order and applies the right handler from this map.",

"reasoning": "The answer explicitly explains that the wrapper component relies on a dispatch table (`renderMap`) and describes how `renderByOrder` processes the children via this table, satisfying the requirement to explain this custom rendering approach."

},

{

"id": "completeness-34075",

"tag": "completeness",

"weight": "critical",

"description": "Identifies `CategoricalChartWrapper` as the returned React class component",

"rationale": "The factory ultimately produces this class and ignoring it misses the entire React integration and lifecycle layer",

"dependent_on": [],

```

    "pass": true,
    "evidence": "\u201cCategoricalChartWrapper (L1020) is the React class component. It is the main chart component.\u201d",
    "reasoning": "The golden answer explicitly names CategoricalChartWrapper as the React class component produced by the factory, satisfying the requirement to identify it as the returned class."
  },
  {
    "id": "completeness-34142",
    "tag": "completeness",
    "weight": "bonus",
    "description": "States that `calculateActiveTickIndex` uses the sorted tick array to perform an efficient left-right comparison scan",
    "rationale": "Specific algorithmic approach used for the nearest neighbour hover detection",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"`tooltipTicksGenerator` ... returns two arrays ... `orderedTooltipTicks` which is sorted by pixel coordinate, enabling `calculateActiveTickIndex` to do an efficient left-right comparison scan to find the nearest tick to the mouse.",
    "reasoning": "The answer explicitly states that `calculateActiveTickIndex` uses the sorted tick array (`orderedTooltipTicks`) to perform an efficient left-right comparison when determining the nearest tick, directly satisfying the criterion."
  },
  {
    "id": "completeness-35755",
    "tag": "completeness",
    "weight": "critical",
    "description": "Mentions that `renderByOrder` walks through the JSX children in order applying the right handler from `renderMap`",

```

"rationale": "This function is important to preserve the DOM order while optionally enforcing that certain component types only render once.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cRather than rendering `{children}` directly, the component uses a dispatch table (`renderMap`). `renderByOrder` walks the JSX children in order and applies the right handler from this map.\u201d",

"reasoning": "The golden answer explicitly states that `renderByOrder` processes the JSX children sequentially and uses `renderMap` to apply the appropriate handler, directly satisfying the criterion."

},

{

"id": "completeness-34107",

"tag": "completeness",

"weight": "penalty",

"description": "Suggests removing or bypassing the Redux `RechartsStoreProvider` wrapper",

"rationale": "Actively breaks the state management context required by the library at the highest level of the component",

"dependent_on": [],

"pass": true,

"evidence": "\u201cThe factory ultimately returns this thin functional wrapper that: 1. Creates a Redux store options (`ChartOptions`) 2. Wraps `CategoricalChartWrapper` in `RechartsStoreProvider`.\u201d",

"reasoning": "The answer merely describes that the wrapper exists and does not suggest removing or bypassing it, so the harmful behavior is absent."

},

{

"id": "completeness-34079",

"tag": "completeness",

```
"weight": "bonus",
  "description": "Mentions that `CategoricalChartWrapper` establishes a mouse
movement speed limit in its constructor",
  "rationale": "Highlights a critical performance and anti crash safe guard built into the
UI layer",
  "dependent_on": [
    "completeness-34075"
  ],
  "pass": true,
  "evidence": "\u201c`constructor` and `componentWillUnmount`: When the chart is
first created, it sets up an ID for cropping the chart area and creates a speed-limit for
mouse movements so the browser doesn't crash when the cursor is moved
around.\u201d",
  "reasoning": "The golden answer explicitly states that the constructor creates a
speed limit for mouse movements, satisfying the requirement."
},
{
  "id": "completeness-34130",
  "tag": "completeness",
  "weight": "bonus",
  "description": "Mentions `calculateActiveTickIndex` as a dependent function called
within the `getTooltipData` execution flow",
  "rationale": "Shows the understanding of the internal dependency chain of the tooltip
coordinate resolution",
  "dependent_on": [],
  "pass": true,
  "evidence": "It first uses the `rangeObj {x, y}` ... passes them to `calculateTooltipPos
... The return value is a number which is passed to `calculateActiveTickIndex` that
calculates the activeIndex closest to the `pos`.",
```



```
    "reasoning": "The golden answer explicitly states that getTooltipData invokes  
calculateActiveTickIndex within its execution flow, satisfying the criterion."  
  },  
  {  
    "id": "completeness-34078",  
    "tag": "completeness",  
    "weight": "bonus",  
    "description": "Details the logic inside `getTooltipContent` for handling duplicate keys  
via `findEntryInArray`,  
    "rationale": "Demonstrates deep inspection of edge cases in the data lookup  
process",  
    "dependent_on": [],  
    "pass": true,  
    "evidence": "The answer states that `getTooltipContent` \"uses `findEntryInArray` to  
look up by `activeLabel` value is duplicates are not allowed or uses `activeIndex` if  
duplicates are allowed or there is no `dataKey`.\"",  
    "reasoning": "This explicitly explains how the function handles duplicate keys: it  
leverages `findEntryInArray` when duplicates are not allowed, and otherwise falls back  
to `activeIndex`, demonstrating the logic for duplicate handling."  
  },  
  {  
    "id": "completeness-34131",  
    "tag": "completeness",  
    "weight": "bonus",  
    "description": "Identifies the specific array method `data.slice()` utilized within  
`getDisplayedData`,  
    "rationale": "Highlights the specific and safe JavaScript operation used to isolate the  
visible data segment",  
    "dependent_on": [],  
    "pass": true,
```

```
"evidence": "This function uses `data.slice()` which is safe.",
"reasoning": "The golden answer explicitly states that `getDisplayedData` uses the
array method `data.slice()`, satisfying the criterion."
},
{
  "id": "completeness-35860",
  "tag": "completeness",
  "weight": "bonus",
  "description": "Mentions that `getMouseInfo` handovers to `handleMouseMove`
`handleMouseEnter` or `handleMouseLeave` handlers after getting the coordinates",
  "rationale": "Raw browser interactions and peripheral device interactions are natively
handled after coordinate computation",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cWhen a click, touch or drag events \u2026 funnels through
\u2026 `getMouseInfo()` \u2026 It then hands it off to the right handler,
`handleMouseMove` `handleMouseEnter` or `handleMouseLeave`.\u201d",
  "reasoning": "The golden answer explicitly states that after `getMouseInfo`
computes the necessary coordinates, it passes control to `handleMouseMove`,
`handleMouseEnter`, or `handleMouseLeave`, satisfying the criterion."
},
{
  "id": "style-34112",
  "tag": "style",
  "weight": "bonus",
  "description": "Formats the explanation using structured hierarchical elements like
markdown headers or numbered lists",
  "rationale": "Given the sheer volume of functions in the target file, structural
formatting is essential for visual parsing",
  "dependent_on": [],
```

```
    "pass": true,
    "evidence": "The response uses markdown headers such as `## Important Helper and Build Functions` and `## `generateCategoricalChart` Factory Function`, along with multiple ordered lists (e.g., `1. Imports and Constants...`, `2. Data Helpers...`).",
    "reasoning": "The explanation is structured with hierarchical markdown headers and numbered lists, fulfilling the requirement for structured formatting."
  },
  {
    "id": "style-34113",
    "tag": "style",
    "weight": "bonus",
    "description": "Presents inline code elements and function names using monospace markdown formatting",
    "rationale": "Maintains strict technical clarity by distinguishing variable/function names from explanation",
    "dependent_on": [],
    "pass": true,
    "evidence": "Examples include `AreaChart`, `generateCategoricalChart`, `getTooltipContent`, `getAxisMapByAxes`, `CategoricalChartWrapper`, `renderMap` and many others presented in backticks.",
    "reasoning": "Function names, files, and code elements are consistently formatted with inline monospace markdown, satisfying the criterion."
  }
],
"passed": 34,
"failed": 0,
"total": 34,
"score_percent": 100.0,
"summary": "Passed 34/34 criteria (100.0%)"
}
```

Golden Answer Eval

```
{
  "pass": true,
  "score_percent": 100.0,
  "earned": 59.0,
  "max_possible": 59.0,
  "critical_fails": [],
  "penalties_triggered": [],
  "summary": "PASS \u2014 2014 golden answer follows rubric (100.0%)",
  "criteria": [
    {
      "id": "reasoning-34071",
      "weight": "penalty",
      "passed": true,
      "score": 0.0
    },
    {
      "id": "reasoning-34055",
      "weight": "bonus",
      "passed": true,
      "score": 1.0
    },
    {
      "id": "reasoning-34049",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34008",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34002",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34059",
```

```
"weight": "bonus",
"passed": true,
"score": 1.0
},
{
  "id": "reasoning-34005",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34057",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34108",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "reasoning-34006",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34051",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34123",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "reasoning-34054",
  "weight": "critical",
  "passed": true,
```

```
"score": 3.0
},
{
  "id": "reasoning-34074",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34141",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34050",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34058",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34143",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34140",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34007",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
}
```

```
{
  "id": "reasoning-33993",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34070",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "completeness-34076",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34075",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34142",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "completeness-35755",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34107",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "completeness-34079",
```

```
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34130",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34078",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34131",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-35860",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "style-34112",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "style-34113",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  }
]
}
```


Test Model

Score

33.9% (hard)

Model Response

=====

RUN 1/3

=====

You're seeing every chart component in `src/chart` "call the same function" because Recharts centralizes all of the cross-cutting chart behaviour inside the factory exported from `src/chart/generateCategoricalChart.tsx`. Instead of re-implementing axis parsing, tooltip state, syncing, accessibility, brushes, legends, portals, etc. for each chart, the library builds a single `CategoricalChartWrapper` class that all categorical/polar charts share, and then parameterizes it with just the pieces that differ per chart. Here's how it's wired together and why.

1. What each chart file actually does

Files such as `AreaChart.tsx`, `BarChart.tsx`, `LineChart.tsx`, `PieChart.tsx`, etc. are extremely thin:

```ts

```
export const BarChart = generateCategoricalChart({
 chartName: 'BarChart',
 GraphicalChild: Bar,
 defaultTooltipEventType: 'axis',
 validateTooltipEventTypes: ['axis', 'item'],
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
```

```

 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
...

(src/chart/BarChart.tsx, lines 11-22)

```

Each one just calls `generateCategoricalChart`` with:

- \* `chartName`` – used for display names, Redux store keys, etc.
- \* `GraphicalChild`` – the primitive that actually renders the data (``Bar``, ``Area``, ``Line``, ``Pie``, ...).
- \* `axisComponents`` – which axis types this chart expects (Cartesian ``XAxis/YAxis``, polar ``angleAxis/radiusAxis``, etc.).
- \* `formatAxisMap`` – helper to massage computed axis maps for the particular coordinate system (``CartesianUtils.formatAxisMap`` vs ``PolarUtils.formatAxisMap``).
- \* Optional knobs like tooltip defaults or chart-specific default props (e.g. ``PieChart`` sets ``layout: 'centric', startAngle: 0, ...`` at lines 11-24 of ``src/chart/PieChart.tsx``).

Everything else—data slicing, state management, syncing, delegates, rendering order—is handled by the generated component.

---

### ### 2. What `generateCategoricalChart`` builds

`generateCategoricalChart`` (entire file ``src/chart/generateCategoricalChart.tsx``) returns a *function component* that wraps a ``CategoricalChartWrapper`` class component in a ``RechartsStoreProvider``. The wrapper:

1. **Derives state from props** (`createDefaultState`, `getDerivedStateFromProps`) — tracking mouse position, brush indexes, tooltip status, cached axis maps, formatted graphical items, etc. (see lines 268-458 and 615-918).
2. **Builds axis maps & offsets** (`getAxisMapByAxes`, `getAxisMapByItems`, `calculateOffset`, `formatAxisMap`) whenever data/layout/brush changes (lines 208-461, 542-693). This is where the `axisComponents` you passed in matter: the factory knows which axis types to extract from children and how to interpret them.
3. **Formats child graphical elements** using the `GraphicalChild`'s static `getComposedData` (lines 694-777). This step injects computed props (band positions, stacked data, animation ids, etc.) so `<Bar />`, `<Line />`, etc. can render correctly without duplicating logic per chart.
4. **Handles interaction plumbing** — tooltips (`getTooltipData`, `handleMouseEnter/Move/Leave`, `handleItemMouseEnter`), syncing multiple charts via `eventCenter`, throttling mouse move, click handlers, accessibility keyboard support, portals for cursors/tooltips, brush updates, legend bounding boxes, etc. (lines 171-692, 779-1314).
5. **Renders** by wrapping children with many context providers and calling `renderByOrder(children, renderMap)` so each supported child type gets either passed through untouched (`renderAsIs`) or cloned with the computed props (`renderGraphicChild`). See the render map around lines 1108-1138.
6. **Finally** the factory returns:

```
``tsx
return function CategoricalChart(props) {
```

```

const options: ChartOptions = { ... };
return (
 <RechartsStoreProvider preloadedState={{ options }} reduxStoreName={props.id ??
chartName}>
 <CategoricalChartWrapper {...props} />
 </RechartsStoreProvider>
);
};
...
(lines 1272-1289)

```

So every “chart component” you import is really “`generateCategoricalChart` with a specific configuration”.

---

### ### 3. Why Recharts does it this way

**\* \*\*Reduces duplication.\*\*** Axes, tooltips, syncing, animations, accessibility, portals, brushes, etc. are identical whether you’re plotting lines, areas, bars, or pies. The factory keeps that logic in one place.

**\* \*\*Keeps charts consistent.\*\*** When tooltip behaviour or axis formatting changes, fixing it in `generateCategoricalChart` automatically updates all categorical charts.

**\* \*\*Still allows specialization.\*\*** Each chart can override defaults (tooltip mode, layout, axis set, GraphicalChild) without rewriting the boilerplate.

**\* \*\*Future direction.\*\*** The file is full of comments about moving away from DOM-based child introspection (“@deprecated this relies on direct DOM access”). Centralizing the

logic makes it easier to migrate to Redux/context-driven state or plain JSX rendering later.

---

### ### 4. How to work with it as a contributor

1. **Understand `CategoricalChartState`` and contexts** (`src/chart/types.ts``). Most shared logic reads/writes `formattedGraphicalItems``, `xAxisMap``, `tooltipTicks``, etc.

2. **When creating a new chart**, add a new file that calls `generateCategoricalChart`` with:

- \* your `GraphicalChild`` component,
- \* the axis components it needs,
- \* a `formatAxisMap`` suited to its coordinate system,
- \* any default props or tooltip rules.

3. **If you need to change core behaviour** (new tooltip interaction, better stacking, improved accessibility), expect to update `generateCategoricalChart.tsx``. That change will automatically propagate to all charts that reference it.

4. **If something really must diverge**, you can extend the configuration options passed into `generateCategoricalChart``, or add chart-specific behaviour inside your `GraphicalChild`` (e.g. Scatter customizing tooltip positions, see lines 352-374 when `displayDefaultTooltip`` handles Scatter).

---

### ### Mental model

Think of each chart component as a *\*scenario\** fed into a common chart engine. The engine:

1. Collects children → builds axis maps → formats graphical items.
2. Manages interactions and shared state.
3. Renders children in the right order, injecting computed props where needed.

Your glass-box view of a “mini world per chart” still holds for the visual layer (each `<Bar />`, `<Line />` still computes its own shapes), but the scaffolding (axes, tooltips, layout, mouse handling) is centralized to avoid re-implementing the same infrastructure a dozen times.

=====

RUN 2/3

=====

In this repo the majority of “normal” Recharts components (AreaChart, BarChart, LineChart, etc.) aren’t hand-written as separate React classes. Instead, they are all thin configuration files that call the same factory, `generateCategoricalChart`, defined in `**src/chart/generateCategoricalChart.tsx**` (starts around line 802). The reason is that all of these charts share identical scaffolding: measuring the SVG container, parsing `<XAxis />` / `<YAxis />`, building brush ranges, synchronizing tooltips, hosting accessibility features, wiring Redux, etc. Rather than duplicating thousands of lines per chart, the project keeps a single “engine” and lets each chart describe only what’s different.

### How a chart file is structured

Take `**src/chart/BarChart.tsx**` (lines 11-22) as an example:

```
``ts
```

```

export const BarChart = generateCategoricalChart({
 chartName: 'BarChart',
 GraphicalChild: Bar,
 defaultTooltipEventType: 'axis',
 validateTooltipEventTypes: ['axis', 'item'],
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
...

```

Each chart passes a configuration object to the generator:

- \* ``chartName`` is only used for display/debugging and store keys.
- \* ``GraphicalChild`` tells the engine which graphical elements to treat as the “data series” it should render. It can be a single component (AreaChart uses ``Area``) or an array (ComposedChart passes ``[Line, Area, Bar, Scatter]``; see `**`src/chart/ComposedChart.tsx` lines 11-19**`).
- \* ``axisComponents`` describes which axis components the chart supports so the generator can discover ``<XAxis>`` / ``<YAxis>`` children and build axis maps.
- \* ``defaultTooltipEventType`` and ``validateTooltipEventTypes`` adjust tooltip UX (Scatter and Pie are ``item``-driven, Bar/Line/Area default to axis-aligned tooltips).
- \* ``formatAxisMap``, ``tooltipPayloadSearcher``, and optional ``defaultProps`` let each chart tweak formatting, tooltip lookup strategy, or default layout (e.g., PieChart/RadarChart set radial defaults; see `**`src/chart/PieChart.tsx` lines 11-27**`).

Because each chart is just “configuration”, they look minimal, yet the generator injects the massive shared implementation.

### What `generateCategoricalChart`` actually does

Inside `src/chart/generateCategoricalChart.tsx`` the generator returns a component called `CategoricalChartWrapper``. Highlights:

1. **Axis discovery & formatting** – Functions `getAxisMapByAxes``, `getAxisMapByItems``, and `getAxisMap`` (lines ~286-470) inspect the children using `findAllByType`` to collect each `<XAxis>`/`<YAxis>`` (or fall back to inferred axes if the user didn’t declare any). They build maps that include computed domains, ticks, scale info, etc. Config entries from `axisComponents`` drive which axis types are processed.
2. **Graphical item formatting** – `getFormatItems`` (lines ~877-1018) loops over every `GraphicalChild`` found in `props.children``, computes band sizes, stacking data, bar positions, etc., and calls each child’s static `getComposedData`` to produce ready-to-render points. That’s how a `<Bar>`` child gets all the coordinates it needs without the chart itself rendering the bars directly. Only children listed in `GraphicalChild`` are treated this way; everything else (grid, reference lines, etc.) is cloned “as-is”.
3. **Shared state machine** – The wrapper component (lines ~1099-1680) owns the full chart lifecycle:
  - \* default and derived state (`createDefaultState``, `getDerivedStateFromProps``) handle brush ranges, tooltip activity, layout changes, etc.;
  - \* event handlers (`handleMouseEnter``, `handleMouseMove``, sync events) keep tooltips, brushing and cross-chart synchronization consistent;



\* contexts (``ChartLayoutContextProvider``, ``TooltipPortalContext``, etc.) and ``RechartsStoreProvider`` wrap the entire surface so Redux slices and hooks can access chart metadata.

4. **Rendering** – ``render()`` (lines ~1578-1664) ultimately renders a ``Surface`` with ``renderByOrder(children, this.renderMap)``. ``renderMap`` dictates how each child type is treated—graphical children call ``renderGraphicChild``, which replaces them with the formatted props computed earlier.

5. **Why not per-chart classes?** Because all categorical charts need the same infrastructure (axes, tooltip orchestration, accessibility manager, syncing), maintaining separate copies would be error-prone. The generator centralizes that logic while still allowing per-chart behavior through its options and through the props of the graphical children themselves.

### ### When a chart really is its “own world”

Not every chart uses this generator. For instance, `**`src/chart/Sankey.tsx`**` is a standalone component with its own layout algorithm, mouse handling, and tooltip context because Sankey doesn’t fit the categorical axis model at all. You’ll recognize such charts because they export a class/function directly rather than calling ``generateCategoricalChart``.

### ### Recap

\* The files in ``src/chart/*.tsx`` that look like ``export const FooChart = generateCategoricalChart({...})`` are just configuration shims.

\* The heavy lifting lives in ``generateCategoricalChart``, which builds a reusable chart wrapper around whatever graphical children you declare.

\* Each chart customizes the shared engine via `GraphicalChild`, supported axes, default props, and tooltip behavior.

\* Truly different chart types (Sankey, Treemap, Sunburst, etc.) have their own implementations because they don't share the categorical axis workflow.

Once you internalize that pattern, contributing becomes easier: you rarely modify `AreaChart.tsx` itself—you tweak shared behavior in `generateCategoricalChart` or adjust the specific graphical child (e.g., `src/cartesian/Bar.tsx`).

=====

RUN 3/3

=====

### Why every chart “just calls a function”

If you open any categorical chart entry point under `src/chart` (e.g. `AreaChart.tsx` lines 4-20), you'll see the same pattern:

```
``ts
export const AreaChart = generateCategoricalChart({
 chartName: 'AreaChart',
 GraphicalChild: Area,
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
``
```

Each chart simply invokes `generateCategoricalChart`` with a configuration object. That helper (defined in `src/chart/generateCategoricalChart.tsx``) returns a fully functional React component class plus a Redux wrapper. The intent is to centralize the heavy, identical plumbing that every Cartesian/polar chart needs: axis resolution, tooltip handling, brushing, syncing, accessibility, portals, etc. Instead of duplicating hundreds of lines per chart, Recharts uses this factory to compose behavior.

### What the generator actually does

`generateCategoricalChart`` (lines 802–1120 and beyond):

1. **Normalizes axis and data state**

- It defines `getFormatItems`` (lines 812–915) which iterates over the “graphical children” (Bar, Line, Pie...) and asks each child to compose its data (`item.type.getComposedData``) with the correct axis info, stacked data, band sizes, etc.
- It builds axis maps via `getAxisMap`` / `getAxisMapByAxes`` / `getAxisMapByItems`` (lines 218–405, 474–532) and formats them with the `formatAxisMap`` function you pass from each chart file.

2. **Calculates layout offsets & tooltip ticks**

- `calculateOffset`` (lines 548–620) figures out how much room axes, legends, brushes take and stores the drawable view box in state.
- `tooltipTicksGenerator`` (lines 700–708) prepares ordered ticks for tooltip interaction.

3. **Provides the state machine**

- `CategoricalChartWrapper`` (lines 1019–2006) is a massive React `Component`` subclass that owns state such as `graphicalItems``, `xAxisMap``, `tooltipTicks``, `activeTooltipIndex``, etc.
- `static getDerivedStateFromProps`` (lines 1183–1333) is responsible for recomputing axis maps, formatted children, and tooltip data when props change.

- Mouse/touch/key handlers (lines 1400–1715) funnel all interaction through shared logic, including sync events (`eventCenter``) when `syncId`` is used.

#### 4. **Sets up React context + store**

- The returned function wraps the class in a `RechartsStoreProvider`` (lines 1993–2006), injecting chart-specific options (tooltip behavior, gaps, stack offset).
- It also wires `ChartLayoutContextProvider``, tooltip contexts, brush contexts, etc. inside `render`` (lines 1786–1990) so that descendants (axes, tooltips, custom components) can consume layout info without prop drilling.

### ### How charts customize the shared implementation

The options you pass to `generateCategoricalChart`` control the behavior:

- **`GraphicalChild``**: either a single component (AreaChart uses `Area`` at line 12) or an array (ComposedChart passes `[Line, Area, Bar, Scatter]`` at lines 11–18). The generator uses `findAllByType(children, GraphicalChild)`` to know which series are “native” to the chart when formatting data.
- **`axisComponents``**: tells the generator which axis React elements to look for. Area/Line charts give `{ axisType: 'xAxis', AxisComp: XAxis }` etc., while Pie/Radar swap in `PolarAngleAxis`` / `PolarRadiusAxis`` (PieChart lines 9-27). FunnelChart passes an empty array because it doesn’t render axes (lines 1-11).
- **`Defaults``**: you can override layout, angles, centers, etc. via `defaultProps`` (e.g. PieChart sets `layout: 'centric', startAngle: 0, endAngle: 360`` on lines 18-26).
- **`Tooltip mode``**: `defaultTooltipEventType`` and `validateTooltipEventTypes`` define whether the chart responds to axis-based vs. item-based tooltips (see ScatterChart lines 11-20 for item-only mode).

So every chart isn't "its own world" because 90% of the lifecycle is identical: load data, build axes, format child props, manage tooltips/brush/legend/portals. The differentiators are captured in that config object.

### ### Exceptions

Not all charts fit this mold. `SunburstChart.tsx` is entirely custom (long file starting line 1) because a sunburst doesn't share Cartesian logic. It still reuses pieces like `RechartsWrapper`, context providers, and the Redux store, but rendering + interaction are bespoke.

### ### Mental model when contributing

When you add or modify a chart:

1. Decide whether it can reuse the categorical generator. If it uses X/Y (or polar) axes and the usual tooltip/brush stack, you probably just pass the right `GraphicalChild` and axis components.
2. If the chart needs unique layout defaults or tooltip behavior, set them in the config. That keeps the shared `CategoricalChartWrapper` logic untouched.
3. For brand-new graphical elements, implement `getComposedData` on the element (e.g. `cartesian/Area.tsx`) so `getFormatItems` can call it and inject the computed props back into the cloned element (lines 884-912).
4. Only build a bespoke chart component (like `SunburstChart`) if the rendering paradigm fundamentally differs.

By understanding `generateCategoricalChart`, you can trace how props flow: chart file → generator config → shared wrapper state → child formatting → cloneElement in `render`. That's why everything "just calls a function"—it's a deliberate abstraction to keep the library consistent and maintainable.

## Evaluation JSON

```
{
 "points": 20.0,
 "maxPoints": 59.0,
 "score_percent_weight": 34,
 "summary_weight": "20.0/59.0 points (34%)",
 "eval_runs": 3,
 "eval_runs_succeeded": 3,
 "per_run_points": [
 17.0,
 26.0,
 17.0
],
 "results": [
 {
 "id": "reasoning-34071",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34055",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34049",
 "weight": "critical",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34008",
 "weight": "critical",
 "pass": false,
 "points": 0.0
 }
]
}
```

```
},
{
 "id": "reasoning-34002",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "reasoning-34059",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34005",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34057",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34108",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34006",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34051",
```

```
"weight": "critical",
"pass": false,
"points": 0.0
},
{
 "id": "reasoning-34123",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34054",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34074",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34141",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34050",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34058",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
}
```



```
},
{
 "id": "reasoning-34143",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34140",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34007",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-33993",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "reasoning-34070",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34076",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "completeness-34075",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
 "id": "completeness-34142",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-35755",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "completeness-34107",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34079",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34130",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34078",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
}
```

```
 },
 {
 "id": "completeness-34131",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "completeness-35860",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "style-34112",
 "weight": "bonus",
 "pass": true,
 "points": 1.0
 },
 {
 "id": "style-34113",
 "weight": "bonus",
 "pass": true,
 "points": 1.0
 }
]
}
```

**model eval score 33.9**

**Difficulty**    **hard**